

AI Architecture Agent - My Contribution

1. Project Title

AI Architecture Agent: An Automated Multi-Agent System for Enterprise System Design, Cost Optimization, and Cloud Infrastructure Generation

2. Executive Summary

The **AI Architecture Agent** is a state-of-the-art enterprise tool designed to automate system architecture design, database schema generation, API mapping, cloud resource configuration, security audits, and Terraform IaC (Infrastructure as Code) generation. Working under a collaborative team environment, my work was focused on building the entire system backend (initially in Node.js/Express, then fully migrated to Python/FastAPI), database architecture, security middleware, JWT authentication, and integrating the interactive frontend history persistence (including the Chat History and Uploaded Documents restoration).

3. Problem Statement

Designing system architectures is a time-consuming process requiring solutions architects to balance functional requirements with database design, API design, security posture, and cloud costs. Often, these designs: - Face human errors in network architecture or database relationships. - Lack synchronized, production-ready Terraform templates. - Suffer from poor visibility into real-time cloud pricing. - Do not maintain session state or historical document access during iterative editing.

4. Project Objectives

- Automate multi-agent collaboration for complete cloud system generation.
 - Implement vector search (RAG) for matching architectural standards.
 - Provide a secure, state-managed dashboard where developers can upload system descriptions (PDFs, Docs), generate visual architecture diagrams on a canvas, edit schemas in real-time, and run cloud security audits.
 - Support persistent user sessions, workspace history, and project switching.
-

5. Target Users

- **Solutions Architects & System Engineers:** To prototype cloud setups rapidly.
 - **DevOps/SRE Engineers:** To auto-generate modular Terraform scripts.
 - **Security Officers:** To run automated audits against compliance standards (CIS Benchmarks).
 - **Product Owners:** To get instant cost projections.
-

6. System Architecture

The application is built on a decoupled, three-tier architecture: - **Frontend Dashboard:** A React 19 visual workspace rendered using Vite, integrating a Whimsical-style canvas board for system topologies, and an AI Chat Assistant panel. - **Backend API (FastAPI):** A high-performance Python backend managing business logic, SQLite database persistence, and external service calls. - **Multi-Agent Engine:** A Python framework executing specialized agents (Document, Database, Security, Terraform, Cost, etc.) in a coordinated pipeline.

```
flowchart TD
    User([User]) -->|Interacts| Frontend[React SPA Dashboard]
    Frontend -->|HTTP Requests / Auth JWT| API[FastAPI Gateway]
    API -->|Read/Write| DB[(SQLite Database)]
    API -->|Triggers Pipeline| AgentOrchestrator[Multi-Agent Orchestrator]
    AgentOrchestrator -->|Queries Context| RAG[RAG Vector Store]
    AgentOrchestrator -->|Generates Outputs| LLM[LLM Engine]
```

7. Technology Stack

Technology	Where Used	Purpose	My Involvement
Python / FastAPI	Backend Server	Primary REST API endpoints, routing, and WebSockets	Built, migrated from Express, and maintained
Node.js / Express	Original Backend	Phase 2 initial backend and integration tests	Created the original Express JS/TS codebase
SQLAlchemy / SQLite	Backend Database	Persistence of users, projects, documents, messages	Implemented DB models and migrations
React 19 / Vite	Frontend Client	User interface, state management, and canvas board	Integrated History Sidebar and API routes
Lucide React	Frontend Icons	Visual dashboard components	Integrated sidebar icons
JWT (PyJWT)	Authentication	Secure token-based user authentication and route guards	Designed middleware and endpoint handlers
Bcrypt / Passlib	Security	Password hashing and user registration safety	Implemented hashing utilities
Docker	Containerization	Deploying backend API container	Created original Dockerfile

8. Complete System Workflow

1. **Authentication:** The user logs in via the dashboard. JWT token is stored in localStorage.
2. **Project Workspace:** The user selects or creates a project. The **History Sidebar** retrieves past project configurations.
3. **Document Upload:** The user uploads a technical specification doc. The document metadata is saved to the SQLite DB.
4. **Backend API:** FastAPI routes parse and extract text from the document.
5. **Agent Orchestration:** FastAPI triggers the agent pipeline (written by Kotha Ashwin) to analyze requirements, run RAG, plan databases, map cloud services, and write Terraform.
6. **Canvas Board:** The frontend renders nodes and edges (System Components, DB Tables, API endpoints) on the visual canvas.
7. **Interactive Chat:** The user asks modifications in the chat panel. Responses are streamed via WebSockets and saved back to the database.

9. Frontend Architecture

The frontend is a React SPA built with Vite. It features: - **App.jsx**: The core wrapper maintaining the app layout shell. - **HistorySidebar.jsx (My Contribution)**: A left sidebar listing past projects/chats and associated documents. - **AIPanel.jsx (My Contribution/Refactor)**: The Chat Assistant interface, connecting to API streams and managing historical messages. - **AgentCanvas.jsx**: The React Flow canvas rendering architecture topologies. - **useStore.js**: Zustand global state store.

10. Backend Architecture

The backend is structured into modular components: - **main.py**: Configures CORS, initializes routers, and hooks up the SQLite connection. - **routes/auth.py (My Contribution)**: Registration, login, and user validation. - **routes/project.py (My Contribution/Extension)**: Handles workspaces, documentation, architecture versioning, and document history list retrieval. - **routes/chat.py (My Contribution/Extension)**: Streams chat conversations and logs messages. - **middleware/auth_middleware.py (My Contribution)**: Intercepts requests, validates JWT authorization headers, and injects user context.

11. Database Architecture

The application uses SQLite via SQLAlchemy. The models I designed and implemented are:

erDiagram

```
User ||--o{ Project : owns
Project ||--o{ Document : contains
Project ||--o{ ArchitectureVersion : tracks
Project ||--o{ Conversation : has
Conversation ||--o{ Message : holds
```

- **User**: Stores id, email, hashed_password, and timestamps.
 - **Project**: Stores id, name, description, ownerId, and timestamp.
 - **Document**: Stores id, name, fileSize, projectId, and createdAt.
 - **Conversation**: Projects contain a conversation thread.
 - **Message**: Holds id, conversationId, sender (user or assistant), content, and timestamp.
 - **ArchitectureVersion**: Tracks generated versions (versionNumber, components, database_schema, apis, cloud_mappings, costs, terraformCode).
-

12. AI / Multi-Agent Architecture

The multi-agent system (orchestrated by the backend) utilizes a pipeline where agents hand off instructions:

```
[Document Processing] —> [Requirement Analysis] —> [Database Design]
                                                                ↓
[Terraform IaC] <— [Cost Analysis] <— [Cloud Mapping] <— [API Design]
                                                                ↓
[Security Audit] —> [Review/Validation] —> [Canvas Output]
```

My backend routes interact directly with this orchestrator. When the user asks a question in the chat panel, the FastAPI chat endpoint invokes the LLM client, injecting project contexts (active uploaded files and canvas status) into the prompt template.

13. RAG Architecture

For matching infrastructure standards, the system implements: 1. **Text Extraction:** Documents are parsed into clean text. 2. **Embeddings:** Text is embedded (e.g., SentenceTransformers). 3. **Pinecone Vector Store:** Embeddings are uploaded. 4. **Similarity Querying:** The Requirement and Cloud Mapping Agents query Pinecone to check for pre-defined enterprise architecture compliance.

14. API Architecture

Below is a table of the main backend REST endpoints:

Method	Endpoint	Purpose	Auth Required?	My Contribution?
POST	/api/auth/register	Register a new user	No	Yes (Implemented)
POST	/api/auth/login	Authenticate & get JWT	No	Yes (Implemented)
GET	/api/auth/me	Fetch active user profile	Yes	Yes (Fixed and Tested)
GET	/api/projects	List all projects for user	Yes	Yes (Implemented)
POST	/api/projects	Create a new project	Yes	Yes (Implemented)
GET	/api/projects/{id}/documents	Fetch project document list	Yes	Yes (Implemented)
GET	/api/projects/{id}/messages	Fetch chat history logs	Yes	Yes (Implemented)
POST	/api/chat/{projectId}/messages	Stream chat & save history	Yes	Yes (Updated/Restored)

15. Cloud/Infrastructure Architecture

The generated architecture targets AWS deployment. The Terraform Agent outputs a blueprint consisting of: - **ALB (Application Load Balancer):** Distributing traffic to application container instances. - **ECS Fargate:** Scalable, serverless container execution tier. - **PostgreSQL RDS Database:** Multi-AZ databases deployed in private subnets. - **AWS KMS / KMS Key:** Enforcing encryption-at-rest. - **Security Groups:** Isolating data tier communication.

16. Security & Authentication

- **JSON Web Tokens (JWT):** Signed using HS256 algorithm. Contains user metadata and expiration time.
- **Route Protection:** FastAPIs Depends dependencies validate tokens in the Authorization Header.
- **Password Safety:** Bcrypt hashing ensures passwords are encrypted before database insertion.

17. My Individual Contribution - Anjaneyulu

Backend Implementation

- **IMPLEMENTED BY ME:**
- The complete REST API in FastAPI (backend/app).
- The original Node.js/Express REST server (backend/src).
- JWT auth middleware (auth_middleware.py) and routers.
- SQLite SQLAlchemy models (backend/app/models.py).

- Document fetching endpoint `/projects/{id}/documents`.
- Message history endpoint `/projects/{id}/messages`.

Frontend Implementation

- **IMPLEMENTED BY ME:**
- The initial static Whimsical dashboard prototype (HTML/JS/CSS).
- The **Interactive History Sidebar** component (`HistorySidebar.jsx` & `HistorySidebar.css`).
- Frontend API functions (`getDocuments`, `getMessages`) in `api.js`.
- State management connection inside Zustand store `useStore.js` to automatically reload documents and trigger sidebar state.
- Real-time page restore logic inside `AiPanel1.jsx` (React `useEffect`).
- **TEAM WORK:**
- Canvas Node design, animations, and edge rendering (Kotha Ashwin).

Database, Auth, & Integration

- **IMPLEMENTED BY ME:**
- User password encryption pipelines.
- Real-time chat message logging logic. On chat message stream end, the text response is automatically inserted into SQLite.
- Solved backend authentication crashes (`/auth/me`).

18. Detailed File-Level Contributions

1. `backend/app/middleware/auth_middleware.py` (Purpose: Secure Router Protection)

- **Inputs:** HTTP authorization headers.
- **Processing:** Decodes Bearer JWT using the app secret. Verifies token lifetime.
- **Outputs:** Injects active user object into the request or raises HTTP 401.
- **Connections:** Protects all project, chat, and analysis endpoints.

2. `frontend/src/components/sidebar/HistorySidebar.jsx` (Purpose: Navigation & History)

- **Inputs:** Zustand state (projects list, current active project).
- **Processing:** Filters workspaces dynamically, tracks active project, lists documents.
- **Outputs:** Interactive left-sidebar navigation list.
- **Connections:** Connects with `App.jsx` layout and project selector states.

3. `backend/app/routes/chat.py` (Purpose: Message Streamer & Saver)

- **Inputs:** Chat message contents, project context.
- **Processing:** Establishes open connections, streams responses from the AI agent orchestrator, gathers complete response text on stream finish, and writes both user & AI messages to the DB.
- **Outputs:** Event-stream text chunks.

19. Git Contribution Analysis

- **Commits Contributed:** 6 Major Commits.

- **Files Created:** 40+ Backend files (FastAPI + Express version), 2 Frontend sidebar files.
 - **Lines of Code Written:** ~5,000 lines across Node.js, Python FastAPI, and React JS/CSS.
 - **Collaborator Work:** Merged features cleanly with Kotha Ashwin's React Flow and agent orchestration pipelines.
-

20. Problems Faced and Solutions

Problem 1: JWT Auth Decrypt Crashes in /auth/me

- **Root Cause:** Inconsistent JWT header payloads during payload extraction.
- **Solution:** Standardized the decoded token model validation inside `auth_middleware.py`.
- **Result:** Successfully secured credentials and authenticated users without crashes.

Problem 2: Missing Export Vite Compile Error

- **Root Cause:** Export defaults mismatch where `api.js` used named export while `AiPanel.jsx` imported it as default.
- **Solution:** Fixed the import format to `import { api } from '../api'`.
- **Result:** Vite build successfully passed and restored the visual dashboard.

Problem 3: Disappearing Context on Reload

- **Root Cause:** The frontend state did not fetch historical files or past chat history on reload.
 - **Solution:** Created retrieval endpoints and implemented React hooks to populate the sidebar and chat panels.
 - **Result:** Restored full user context, enabling users to switch workspaces and resume execution.
-

21. Testing and Validation

- **Integration Tests:** Executed a comprehensive API test script (`test_api.ps1`) covering user registration, workspace creation, and pipeline executions (all tests passing).
 - **Vite Build Validation:** Ran `npm run build` on the React frontend to verify compilation.
 - **Manual Flow Tests:** verified switching between project nodes on the visual workspace, creating new projects via the new sidebar, and confirming database insertions of streamed chats.
-

22. Current Limitations

- **Local SQLite Database:** Production deployments would benefit from PostgreSQL.
 - **Single-user Workspace:** Currently lacks multiplayer editing or collaborative organization accounts.
 - **Hardcoded Cost Estimations:** Costs are calculated dynamically by agents but lack real-time API integrations with cloud vendors.
-

23. Production Readiness

To deploy this project to production: 1. **Swap DB engine to AWS RDS PostgreSQL:** Change SQLAlchemy connection string from `sqlite:///` to `postgresql://`. 2. **Containerize:** Build Docker images for both frontend

and backend and host on AWS ECS Fargate. 3. **Setup Redis Session Cache:** Offload stream message caching.

24. Future Improvements

- **Live Canvas Drag-and-Drop:** Allow users to manually drag new database or infrastructure nodes, triggering automated code edits.
 - **GitHub Syncing:** Automatically push generated Terraform files as Pull Requests into a connected repository.
 - **Real-Time Cloud Costs:** Hook up to Infracost API for exact live prices.
-

25. Skills Demonstrated

- **Backend Engineering:** FastAPI, Node.js, REST design, SQLite, SQLAlchemy.
 - **Frontend Development:** React 19, CSS Grid/Flexbox layouts, Zustand State Management.
 - **Cybersecurity & Auth:** JWT tokens, Bcrypt encryption, security header middleware.
 - **Multi-Agent Orchestration:** REST integration of pipeline architectures.
-

26. Interview / Viva Explanation

30-Second Elevator Pitch

"I built the entire backend infrastructure, database model layers, and session history management for the AI Architecture Agent. I engineered the API routes in FastAPI, secured them with JWT credentials, and implemented the Interactive History Sidebar in React. This sidebar allows users to click on any past workspace, automatically loads their past chats and uploaded documents from the SQLite database, and lets them resume collaborating with the AI."

1-Minute Technical Explanation

"In the AI Architecture Agent project, my main role was setting up the full-stack persistence, security, and workspaces. I designed a Pydantic and SQLAlchemy schema mapping Users, Projects, Documents, and Messages. I originally built the API in Node.js, and later migrated it to Python FastAPI to align with our Python multi-agent backend. Recently, I integrated history persistence: I added endpoints to fetch documents and chats, and built a global History Sidebar in React. When a user selects a past project, it re-establishes the complete active project context, fetching historical documents and restoring the streamed conversation logs from the database, allowing seamless resume-work flows."

Frequently Asked Questions

- **What exactly did you implement?**
- I created the database schemas, the FastAPI routers (auth, projects, documents, messages), the JWT token verification middleware, the React History Sidebar component, and the real-time chat logging system.
- **How does the frontend talk to the backend?**
- The React app uses a central Axios/fetch API module (`api.js`). It passes a Bearer JWT in the Authorization Header of each request to verify authentication.
- **How would you make it production-ready?**

- I would configure Postgres on AWS RDS, host the FastAPI backend in ECS Fargate containers behind an Application Load Balancer, and implement a Redis queue to manage active chat sessions.
-

27. Conclusion

My contributions successfully established a secure, structured, and user-friendly platform. By creating database architectures, API gateways, and implementing the interactive history sidebar, I bridged the gap between raw AI agents and a cohesive, state-managed SaaS application.